

C++ STL Complete Template Reference

All Containers, Adapters, Iterators, Algorithms & Their Variations

Contents

1. Sequence Containers	3
1.1 <code>std::vector</code>	3
1.2 <code>std::deque</code>	3
1.3 <code>std::list</code> (doubly linked list)	4
1.4 <code>std::forward_list</code> (singly linked list)	4
1.5 <code>std::array</code> (fixed-size)	5
1.6 <code>std::string</code> (character sequence container)	5
2. Container Adapters	7
2.1 <code>std::stack</code>	7
2.2 <code>std::queue</code>	7
2.3 <code>std::priority_queue</code>	7
3. Associative Containers (ordered, tree-based)	8
3.1 <code>std::set</code>	8
3.2 <code>std::multiset</code>	8
3.3 <code>std::map</code>	8
3.4 <code>std::multimap</code>	9
4. Unordered Associative Containers (hash-based)	9
4.1 <code>std::unordered_set</code>	9
4.2 <code>std::unordered_multiset</code>	10
4.3 <code>std::unordered_map</code>	10
4.4 <code>std::unordered_multimap</code>	10
5. Utility / Value Templates	11
5.1 <code>std::pair</code>	11
5.2 <code>std::tuple</code>	11
5.3 <code>std::optional</code> (C++17)	11
5.4 <code>std::variant</code> (C++17)	11
5.5 <code>std::any</code> (C++17)	12
5.6 Smart pointers (memory management templates)	12
6. Iterators	13
7. <code><algorithm></code> — All Major Algorithm Families	14
7.1 Non-modifying sequence operations	14
7.2 Modifying sequence operations	14
7.3 Partitioning operations	15
7.4 Sorting operations	15
7.5 Binary search operations (on sorted ranges)	15
7.6 Set operations (on sorted ranges)	15
7.7 Heap operations	15
7.8 Min/max operations	15

7.9 Comparison operations	16
7.10 Permutation operations	16
7.11 Numeric operations (<numeric>)	16
8. Function Objects & <functional>	17
9. <type_traits> — Metaprogramming Templates (selected)	18
10. Ranges (<ranges>, C++20) — Modern View-Based Style	18
11. Quick Selection Guide	18
12. std::bitset (fixed-size bit sequence)	20

1. Sequence Containers

1.1 std::vector

```
#include <vector>

// --- Template signature ---
template<
    class T,
    class Allocator = std::allocator<T>
> class vector;

// --- Construction variations ---
std::vector<int> v1; // empty
std::vector<int> v2(10); // 10 zero-initialized elements
std::vector<int> v3(10, 5); // 10 elements, each = 5
std::vector<int> v4{1, 2, 3, 4}; // initializer list
std::vector<int> v5(v4); // copy constructor
std::vector<int> v6(std::move(v4)); // move constructor
std::vector<int> v7(v4.begin(), v4.end()); // range constructor
std::vector<int> v8(v4, std::allocator<int>()); // with allocator
int arr[] = {1, 2, 3};
std::vector<int> v9(std::begin(arr), std::end(arr));

// --- Element access ---
v1.at(0); v1[0]; v1.front(); v1.back(); v1.data();

// --- Capacity ---
v1.empty(); v1.size(); v1.max_size(); v1.reserve(100);
v1.capacity(); v1.shrink_to_fit();

// --- Modifiers ---
v1.clear();
v1.insert(v1.begin(), 5);
v1.insert(v1.begin(), 3, 5);
v1.insert(v1.begin(), {1, 2, 3});
v1.emplace(v1.begin(), 5);
v1.erase(v1.begin());
v1.erase(v1.begin(), v1.end());
v1.push_back(5); v1.emplace_back(5);
v1.pop_back();
v1.resize(20); v1.resize(20, 1);
v1.swap(v2);

// --- Iterators ---
v1.begin(); v1.end(); v1.cbegin(); v1.cend();
v1.rbegin(); v1.rend(); v1.crbegin(); v1.crend();

// --- Special ---
std::vector<bool> vb{true, false}; // bit-packed specialization
```

1.2 std::deque

```
#include <deque>

template<
    class T,
```

```

    class Allocator = std::allocator<T>
> class deque;

std::deque<int> d1;
std::deque<int> d2(10, 5);
std::deque<int> d3{1,2,3};
std::deque<int> d4(d3.begin(), d3.end());

d1.push_front(1); d1.emplace_front(1);
d1.push_back(1); d1.emplace_back(1);
d1.pop_front(); d1.pop_back();
d1.at(0); d1[0]; d1.front(); d1.back();
d1.insert(d1.begin(), 5);
d1.erase(d1.begin());
d1.resize(5); d1.shrink_to_fit();

```

1.3 std::list (doubly linked list)

```

#include <list>

template<
    class T,
    class Allocator = std::allocator<T>
> class list;

std::list<int> l1;
std::list<int> l2(10, 5);
std::list<int> l3{1,2,3};

l1.push_front(1); l1.push_back(1);
l1.emplace_front(1); l1.emplace_back(1);
l1.pop_front(); l1.pop_back();

// list-specific algorithms (member functions, more efficient than <algorithm>)
l1.merge(l2); // merge sorted lists
l1.merge(l2, std::greater<int>());
l1.splice(l1.begin(), l2); // splice whole list
l1.splice(l1.begin(), l2, l2.begin());
l1.remove(5); // remove by value
l1.remove_if([](int x){ return x < 0; });
l1.reverse();
l1.unique(); // remove consecutive duplicates
l1.unique([](int a, int b){ return a==b; });
l1.sort(); // list has its own sort (stable, O(n log n))
l1.sort(std::greater<int>());

```

1.4 std::forward_list (singly linked list)

```

#include <forward_list>

template<
    class T,
    class Allocator = std::allocator<T>
> class forward_list;

std::forward_list<int> fl1;

```

```

std::forward_list<int> fl2{1,2,3};

fl1.push_front(1); fl1.emplace_front(1); fl1.pop_front();
fl1.front();
fl1.insert_after(fl1.before_begin(), 5);
fl1.emplace_after(fl1.begin(), 5);
fl1.erase_after(fl1.begin());
fl1.merge(fl2);
fl1.splice_after(fl1.begin(), fl2);
fl1.remove(5); fl1.remove_if([](int x){return x<0;});
fl1.reverse(); fl1.unique(); fl1.sort();

```

1.5 std::array (fixed-size)

```

#include <array>

template<
    class T,
    std::size_t N
> struct array;

std::array<int, 5> a1{};
std::array<int, 5> a2{1,2,3,4,5};

a1.at(0); a1[0]; a1.front(); a1.back(); a1.data();
a1.size(); a1.max_size(); a1.empty();
a1.fill(0);
a1.swap(a2);

```

1.6 std::string (character sequence container)

```

#include <string>

template<
    class CharT,
    class Traits = std::char_traits<CharT>,
    class Allocator = std::allocator<CharT>
> class basic_string;

using string      = basic_string<char>;
using wstring     = basic_string<wchar_t>;
using u16string   = basic_string<char16_t>;
using u32string   = basic_string<char32_t>;

std::string s1;
std::string s2("hello");
std::string s3(5, 'x');
std::string s4(s2, 1, 3);           // substring copy
std::string s5{"hello"};
std::string s6 = s2 + s3;

s1.substr(1, 3); s1.find("lo"); s1.rfind("l");
s1.find_first_of("aeiou"); s1.find_last_of("aeiou");
s1.find_first_not_of("abc"); s1.find_last_not_of("abc");
s1.append("x"); s1 += "x"; s1.push_back('x');
s1.insert(0, "x"); s1.erase(0,1); s1.replace(0,1,"y");

```

```
s1.compare(s2); s1.c_str(); s1.data();  
s1.starts_with("he"); s1.ends_with("lo"); // C++20  
s1.contains("ell"); // C++23
```

2. Container Adapters

2.1 std::stack

```
#include <stack>

template<
    class T,
    class Container = std::deque<T>
> class stack;

std::stack<int> s1;
std::stack<int> s2(std::deque<int>{1,2,3});
std::stack<int, std::vector<int>> s3;    // backed by vector
std::stack<int, std::list<int>> s4;     // backed by list

s1.push(1); s1.emplace(1); s1.pop();
s1.top(); s1.empty(); s1.size();
```

2.2 std::queue

```
#include <queue>

template<
    class T,
    class Container = std::deque<T>
> class queue;

std::queue<int> q1;
std::queue<int, std::list<int>> q2;    // backed by list

q1.push(1); q1.emplace(1); q1.pop();
q1.front(); q1.back(); q1.empty(); q1.size();
```

2.3 std::priority_queue

```
#include <queue>

template<
    class T,
    class Container = std::vector<T>,
    class Compare = std::less<typename Container::value_type>
> class priority_queue;

std::priority_queue<int> pq1;                                // max-heap
std::priority_queue<int, std::vector<int>, std::greater<int>> pq2; // min-heap

// custom comparator (lambda type must be spelled out)
auto cmp = [](int a, int b){ return a > b; };
std::priority_queue<int, std::vector<int>, decltype(cmp)> pq3(cmp);

pq1.push(1); pq1.emplace(1); pq1.pop();
pq1.top(); pq1.empty(); pq1.size();
```

3. Associative Containers (ordered, tree-based)

3.1 std::set

```
#include <set>

template<
    class Key,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<Key>
> class set;

std::set<int> s1;
std::set<int> s2{5,3,1,4};           // sorted automatically
std::set<int, std::greater<int>> s3; // descending order
std::set<int> s4(s2.begin(), s2.end());

s1.insert(5); s1.emplace(5);
auto [it, ok] = s1.insert(5);       // returns pair<iterator, bool>
s1.insert({1,2,3});
s1.erase(5); s1.erase(s1.begin());
s1.find(5); s1.count(5);             // 0 or 1
s1.contains(5);                     // C++20
s1.lower_bound(5); s1.upper_bound(5);
s1.equal_range(5);
```

3.2 std::multiset

```
#include <set>

template<
    class Key,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<Key>
> class multiset;

std::multiset<int> ms1{1,1,2,3,3,3}; // duplicates allowed
ms1.insert(3);                      // always succeeds, returns iterator
ms1.count(3);                       // can be > 1
ms1.erase(3);                      // erases ALL matching 3
ms1.erase(ms1.find(3));             // erase just one occurrence
```

3.3 std::map

```
#include <map>

template<
    class Key,
    class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T>>
> class map;

std::map<std::string, int> m1;
std::map<std::string, int> m2{{"a",1}, {"b",2}};
std::map<std::string, int, std::greater<std::string>> m3; // reverse key order
```

```

m1["key"] = 5; // insert-or-update, default constructs if absent
m1.at("key"); // throws if absent
m1.insert({"key", 5});
m1.insert(std::make_pair("key", 5));
m1.emplace("key", 5);
m1.try_emplace("key", 5); // C++17: no-op if key exists
m1.insert_or_assign("key", 5); // C++17
m1.erase("key");
m1.find("key"); m1.count("key"); m1.contains("key"); // contains: C++20
for (auto& [k, v] : m1) { /* structured bindings, C++17 */ }
m1.lower_bound("key"); m1.upper_bound("key"); m1.equal_range("key");

```

3.4 std::multimap

```

#include <map>

template<
    class Key,
    class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T>>
> class multimap;

std::multimap<std::string, int> mm1;
mm1.insert({"a", 1});
mm1.insert({"a", 2}); // both "a" entries coexist
// NOTE: multimap has NO operator[] or at()
auto range = mm1.equal_range("a"); // iterate all values for "a"
for (auto it = range.first; it != range.second; ++it) it->second;

```

4. Unordered Associative Containers (hash-based)

4.1 std::unordered_set

```

#include <unordered_set>

template<
    class Key,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<Key>
> class unordered_set;

std::unordered_set<int> us1;
std::unordered_set<int> us2{1,2,3};

// custom hash + equality for user-defined types
struct PointHash {
    std::size_t operator()(const std::pair<int,int>& p) const {
        return std::hash<int>()(p.first) ^ (std::hash<int>()(p.second) << 1);
    }
};

std::unordered_set<std::pair<int,int>, PointHash> us3;

us1.insert(5); us1.emplace(5);

```

```
us1.find(5); us1.count(5); us1.contains(5);
us1.bucket_count(); us1.load_factor(); us1.max_load_factor();
us1.rehash(100); us1.reserve(100);
```

4.2 std::unordered_multiset

```
#include <unordered_set>

template<
    class Key,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<Key>
> class unordered_multiset;

std::unordered_multiset<int> ums1{1,1,2,3};    // duplicates allowed, no order
```

4.3 std::unordered_map

```
#include <unordered_map>

template<
    class Key,
    class T,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<std::pair<const Key, T>>
> class unordered_map;

std::unordered_map<std::string, int> um1;
um1["key"] = 5;
um1.insert({"key", 5});
um1.emplace("key", 5);
um1.try_emplace("key", 5);
um1.insert_or_assign("key", 5);
um1.find("key"); um1.count("key"); um1.contains("key");
um1.bucket("key"); um1.bucket_count();
```

4.4 std::unordered_multimap

```
#include <unordered_map>

template<
    class Key,
    class T,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<std::pair<const Key, T>>
> class unordered_multimap;

std::unordered_multimap<std::string, int> umm1;
umm1.insert({"a", 1});
umm1.insert({"a", 2});
auto range = umm1.equal_range("a");
```

5. Utility / Value Templates

5.1 std::pair

```
#include <utility>

template<class T1, class T2> struct pair;

std::pair<int, std::string> p1;
std::pair<int, std::string> p2(1, "a");
std::pair<int, std::string> p3 = std::make_pair(1, "a");
auto p4 = std::make_pair(1, "a");           // type deduced
p1.first; p1.second;
std::get<0>(p1); std::get<1>(p1);
auto [a, b] = p1;                          // structured bindings (C++17)
```

5.2 std::tuple

```
#include <tuple>

template<class... Types> class tuple;

std::tuple<int, double, std::string> t1;
std::tuple<int, double, std::string> t2(1, 2.5, "x");
auto t3 = std::make_tuple(1, 2.5, "x");
auto t4 = std::tie(a, b, c);               // tuple of references
std::get<0>(t1); std::get<double>(t1);     // by index or unique type
std::tuple_size<decltype(t1)>::value;
std::tuple_cat(t1, t2);                    // concatenate tuples
auto [x, y, z] = t1;                       // structured bindings
```

5.3 std::optional (C++17)

```
#include <optional>

template<class T> class optional;

std::optional<int> o1;                      // empty
std::optional<int> o2 = 5;
std::optional<int> o3 = std::nullopt;
std::optional<int> o4 = std::make_optional(5);

if (o2) { *o2; o2.value(); }
o2.value_or(0);
o2.has_value();
o2.reset();
o2.emplace(10);
```

5.4 std::variant (C++17)

```
#include <variant>

template<class... Types> class variant;

std::variant<int, double, std::string> v1;
std::variant<int, double, std::string> v2 = "hello";
```

```

std::get<std::string>(v2);
std::get<2>(v2);
std::get_if<int>(&v2);
v2.index();
std::holds_alternative<std::string>(v2);
std::visit([](auto&& val){ /* generic lambda visitor */ }, v2);

```

5.5 std::any (C++17)

```

#include <any>

class any;    // not a template itself, holds any type

std::any a1 = 5;
a1 = std::string("hi");
std::any_cast<std::string>(a1);
a1.has_value(); a1.type();

```

5.6 Smart pointers (memory management templates)

```

#include <memory>

template<class T, class Deleter = std::default_delete<T>> class unique_ptr;
template<class T> class shared_ptr;
template<class T> class weak_ptr;

std::unique_ptr<int> u1 = std::make_unique<int>(5);
std::unique_ptr<int[]> u2 = std::make_unique<int[]>(10);    // array form
std::unique_ptr<int, void(*)(int*)> u3(new int(5), [](int* p){ delete p; });

std::shared_ptr<int> s1 = std::make_shared<int>(5);
std::shared_ptr<int> s2(s1);                                // shared ownership
std::weak_ptr<int> w1 = s1;                                  // non-owning observer
if (auto locked = w1.lock()) { /* use locked */ }

```

6. Iterators

```
#include <iterator>

// --- Iterator category tags ---
struct input_iterator_tag {};
struct output_iterator_tag {};
struct forward_iterator_tag : public input_iterator_tag {};
struct bidirectional_iterator_tag : public forward_iterator_tag {};
struct random_access_iterator_tag : public bidirectional_iterator_tag {};
struct contiguous_iterator_tag : public random_access_iterator_tag {}; // C++20

// --- Iterator adaptors ---
template<class Iter> class reverse_iterator;
template<class Container> class back_insert_iterator;
template<class Container> class front_insert_iterator;
template<class Container> class insert_iterator;
template<class T> class istream_iterator;
template<class T> class ostream_iterator;
template<class Iter> class move_iterator;

std::vector<int> v{1,2,3};

std::reverse_iterator<std::vector<int>::iterator> rit(v.end());
auto bii = std::back_inserter(v); // wraps back_insert_iterator
auto fii = std::front_inserter(v); // wraps front_insert_iterator (needs push_front)
auto ii = std::inserter(v, v.begin()); // wraps insert_iterator

std::istream_iterator<int> in_it(std::cin);
std::istream_iterator<int> in_end;
std::ostream_iterator<int> out_it(std::cout, " ");

auto mit = std::make_move_iterator(v.begin());

// --- Free functions ---
std::advance(rit, 2);
std::distance(v.begin(), v.end());
std::next(v.begin(), 2);
std::prev(v.end(), 2);
std::begin(v); std::end(v); std::cbegin(v); std::cend(v);
std::rbegin(v); std::rend(v);
std::size(v); std::empty(v); std::data(v);
```

7. <algorithm> — All Major Algorithm Families

7.1 Non-modifying sequence operations

```
#include <algorithm>

std::all_of(v.begin(), v.end(), [](int x){ return x>0; });
std::any_of(v.begin(), v.end(), [](int x){ return x>0; });
std::none_of(v.begin(), v.end(), [](int x){ return x>0; });
std::for_each(v.begin(), v.end(), [](int& x){ x *= 2; });
std::for_each_n(v.begin(), 3, [](int& x){ x *= 2; });    // C++17
std::count(v.begin(), v.end(), 5);
std::count_if(v.begin(), v.end(), [](int x){ return x>0; });
std::mismatch(v.begin(), v.end(), v2.begin());
std::find(v.begin(), v.end(), 5);
std::find_if(v.begin(), v.end(), [](int x){ return x>0; });
std::find_if_not(v.begin(), v.end(), [](int x){ return x>0; });
std::find_end(v.begin(), v.end(), sub.begin(), sub.end());
std::find_first_of(v.begin(), v.end(), s.begin(), s.end());
std::adjacent_find(v.begin(), v.end());
std::search(v.begin(), v.end(), sub.begin(), sub.end());
std::search_n(v.begin(), v.end(), 3, 5);    // find 3 consecutive 5's
```

7.2 Modifying sequence operations

```
std::copy(v.begin(), v.end(), dest.begin());
std::copy_if(v.begin(), v.end(), dest.begin(), [](int x){return x>0;});
std::copy_n(v.begin(), 3, dest.begin());
std::copy_backward(v.begin(), v.end(), dest.end());
std::move(v.begin(), v.end(), dest.begin());    // algorithm, not std::move<T>
std::move_backward(v.begin(), v.end(), dest.end());
std::fill(v.begin(), v.end(), 0);
std::fill_n(v.begin(), 3, 0);
std::transform(v.begin(), v.end(), dest.begin(), [](int x){ return x*2; });
std::transform(v.begin(), v.end(), v2.begin(), dest.begin(),
    [](int a, int b){ return a+b; });    // binary version
std::generate(v.begin(), v.end(), [](){ return rand(); });
std::generate_n(v.begin(), 3, [](){ return rand(); });
std::remove(v.begin(), v.end(), 5);    // returns new logical end
std::remove_if(v.begin(), v.end(), [](int x){return x<0;});
std::remove_copy(v.begin(), v.end(), dest.begin(), 5);
std::remove_copy_if(v.begin(), v.end(), dest.begin(), [](int x){return x<0;});
std::replace(v.begin(), v.end(), 5, 10);
std::replace_if(v.begin(), v.end(), [](int x){return x<0;}, 0);
std::replace_copy(v.begin(), v.end(), dest.begin(), 5, 10);
std::replace_copy_if(v.begin(), v.end(), dest.begin(), [](int x){return x<0;}, 0);
std::swap(a, b);
std::swap_ranges(v.begin(), v.end(), v2.begin());
std::iter_swap(v.begin(), v2.begin());
std::reverse(v.begin(), v.end());
std::reverse_copy(v.begin(), v.end(), dest.begin());
std::rotate(v.begin(), v.begin()+2, v.end());
std::rotate_copy(v.begin(), v.begin()+2, v.end(), dest.begin());
std::shift_left(v.begin(), v.end(), 2);    // C++20
std::shift_right(v.begin(), v.end(), 2);    // C++20
std::sample(v.begin(), v.end(), dest.begin(), 3, gen);    // C++17 random sample
std::shuffle(v.begin(), v.end(), gen);    // needs URNG e.g. std::mt19937
```

```
std::unique(v.begin(), v.end());
std::unique(v.begin(), v.end(), [](int a,int b){return a==b;});
std::unique_copy(v.begin(), v.end(), dest.begin());
```

7.3 Partitioning operations

```
std::is_partitioned(v.begin(), v.end(), pred);
std::partition(v.begin(), v.end(), pred);
std::partition_copy(v.begin(), v.end(), out_true.begin(), out_false.begin(), pred);
std::stable_partition(v.begin(), v.end(), pred);
std::partition_point(v.begin(), v.end(), pred);
```

7.4 Sorting operations

```
std::sort(v.begin(), v.end());
std::sort(v.begin(), v.end(), std::greater<int>());
std::sort(v.begin(), v.end(), [](int a,int b){ return a>b; });
std::stable_sort(v.begin(), v.end());
std::partial_sort(v.begin(), v.begin()+3, v.end());
std::partial_sort_copy(v.begin(), v.end(), dest.begin(), dest.end());
std::is_sorted(v.begin(), v.end());
std::is_sorted_until(v.begin(), v.end());
std::nth_element(v.begin(), v.begin()+3, v.end()); // partial ordering around nth
```

7.5 Binary search operations (on sorted ranges)

```
std::lower_bound(v.begin(), v.end(), 5);
std::upper_bound(v.begin(), v.end(), 5);
std::binary_search(v.begin(), v.end(), 5);
std::equal_range(v.begin(), v.end(), 5);
```

7.6 Set operations (on sorted ranges)

```
std::merge(v1.begin(), v1.end(), v2.begin(), v2.end(), dest.begin());
std::inplace_merge(v.begin(), v.begin()+3, v.end());
std::includes(v1.begin(), v1.end(), v2.begin(), v2.end());
std::set_union(v1.begin(), v1.end(), v2.begin(), v2.end(), dest.begin());
std::set_intersection(v1.begin(), v1.end(), v2.begin(), v2.end(), dest.begin());
std::set_difference(v1.begin(), v1.end(), v2.begin(), v2.end(), dest.begin());
std::set_symmetric_difference(v1.begin(), v1.end(), v2.begin(), v2.end(), dest.begin());
```

7.7 Heap operations

```
std::make_heap(v.begin(), v.end());
std::push_heap(v.begin(), v.end());
std::pop_heap(v.begin(), v.end());
std::sort_heap(v.begin(), v.end());
std::is_heap(v.begin(), v.end());
std::is_heap_until(v.begin(), v.end());
```

7.8 Min/max operations

```

std::min(a, b);
std::min(a, b, cmp);
std::min({1,2,3}); // initializer_list version
std::max(a, b); std::max({1,2,3});
std::minmax(a, b); // returns pair
std::minmax({1,2,3});
std::min_element(v.begin(), v.end());
std::max_element(v.begin(), v.end());
std::minmax_element(v.begin(), v.end());
std::clamp(v, lo, hi); // C++17

```

7.9 Comparison operations

```

std::equal(v1.begin(), v1.end(), v2.begin());
std::equal(v1.begin(), v1.end(), v2.begin(), v2.end()); // 4-iterator overload
std::lexicographical_compare(v1.begin(), v1.end(), v2.begin(), v2.end());
std::lexicographical_compare_three_way(v1.begin(), v1.end(), v2.begin(), v2.end()); // C++20

```

7.10 Permutation operations

```

std::is_permutation(v1.begin(), v1.end(), v2.begin());
std::next_permutation(v.begin(), v.end());
std::prev_permutation(v.begin(), v.end());

```

7.11 Numeric operations (<numeric>)

```

#include <numeric>

std::accumulate(v.begin(), v.end(), 0);
std::accumulate(v.begin(), v.end(), 0, [](int a,int b){ return a+b; });
std::reduce(v.begin(), v.end()); // C++17, parallel-friendly
std::inner_product(v1.begin(), v1.end(), v2.begin(), 0);
std::transform_reduce(v1.begin(), v1.end(), v2.begin(), 0); // C++17
std::partial_sum(v.begin(), v.end(), dest.begin());
std::inclusive_scan(v.begin(), v.end(), dest.begin()); // C++17
std::exclusive_scan(v.begin(), v.end(), dest.begin(), 0); // C++17
std::transform_inclusive_scan(v.begin(), v.end(), dest.begin(), std::plus<>(), [](int x){return x;});
std::transform_exclusive_scan(v.begin(), v.end(), dest.begin(), 0, std::plus<>(), [](int x){return x;});
std::adjacent_difference(v.begin(), v.end(), dest.begin());
std::iota(v.begin(), v.end(), 0); // fill with 0,1,2,3...
std::gcd(12, 18); std::lcm(4, 6); // C++17
std::midpoint(a, b); // C++20

```

8. Function Objects & <functional>

```
#include <functional>

// --- Arithmetic function objects ---
std::plus<T>();      std::minus<T>();      std::multiplies<T>();
std::divides<T>();   std::modulus<T>();    std::negate<T>();
// transparent versions (C++14), deduce operand types:
std::plus<>();        std::minus<>();        std::multiplies<>();

// --- Comparison function objects ---
std::equal_to<T>();   std::not_equal_to<T>();
std::greater<T>();    std::less<T>();
std::greater_equal<T>(); std::less_equal<T>();
std::greater<>();      std::less<>();          // transparent

// --- Logical function objects ---
std::logical_and<T>(); std::logical_or<T>(); std::logical_not<T>();

// --- Bitwise function objects ---
std::bit_and<T>();    std::bit_or<T>();    std::bit_xor<T>();    std::bit_not<T>();

// --- std::function: type-erased callable wrapper ---
template<class Signature> class function;    // e.g. function<int(int,int)>

std::function<int(int,int)> f1 = [](int a,int b){ return a+b; };
std::function<void()> f2 = someFreeFunction;
std::function<int(int)> f3 = std::bind(std::plus<int>(), 10, std::placeholders::_1);

// --- std::bind ---
using namespace std::placeholders;
auto bound = std::bind(func, _1, 42, _2);

// --- std::hash ---
template<class Key> struct hash;
std::hash<std::string>()("hello");

// --- std::reference_wrapper ---
template<class T> class reference_wrapper;
int x = 5;
std::reference_wrapper<int> rw = std::ref(x);
auto crw = std::cref(x);

// --- std::mem_fn ---
struct Foo { int bar(int) { return 1; } };
auto mf = std::mem_fn(&Foo::bar);

// --- std::invoke (C++17) ---
std::invoke(f1, 1, 2);
std::invoke(&Foo::bar, foo_instance, 5);

// --- Transparent comparator idiom for heterogeneous lookup ---
std::map<std::string, int, std::less<>> transparent_map;
```

9. <type_traits> — Metaprogramming Templates (selected)

```
#include <type_traits>

std::is_integral<T>::value;          std::is_integral_v<T>;      // _v suffix = C++17
std::is_floating_point_v<T>;
std::is_pointer_v<T>;               std::is_reference_v<T>;
std::is_const_v<T>;                 std::is_same_v<T, U>;
std::is_base_of_v<Base, Derived>;
std::is_convertible_v<From, To>;
std::enable_if<Cond, T>::type;      std::enable_if_t<Cond, T>;
std::conditional<Cond, T, F>::type; std::conditional_t<Cond, T, F>;
std::remove_reference_t<T>;          std::remove_const_t<T>;
std::add_pointer_t<T>;               std::decay_t<T>;
std::void_t<...>;                    // C++17 detection idiom
if constexpr (std::is_integral_v<T>) { /* compile-time branch, C++17 */ }
```

10. Ranges (<ranges>, C++20) — Modern View-Based Style

```
#include <ranges>
#include <algorithm>

namespace views = std::views;
namespace ranges = std::ranges;

std::vector<int> v{1,2,3,4,5,6,7,8};

auto evens = v | views::filter([](int x){ return x % 2 == 0; });
auto doubled = v | views::transform([](int x){ return x * 2; });
auto pipeline = v | views::filter([](int x){return x>2;})
                | views::transform([](int x){return x*x;})
                | views::take(3);

views::iota(1, 10);           // lazy range 1..9
views::reverse(v);
views::drop(v, 2);
views::take(v, 3);
views::keys(some_map);        // view of map keys
views::values(some_map);      // view of map values
views::join(nested_ranges);
views::split(str, ' ');

ranges::sort(v);
ranges::for_each(v, [](int x){});
ranges::find(v, 5);
ranges::copy(v, dest.begin());
ranges::all_of(v, pred);
```

11. Quick Selection Guide

Need	Container
Fast random access, back growth	vector
Fast front/back growth	deque
Fast insert/erase in middle	list / forward_list

Need	Container
Fixed size, stack-allocated	<code>array</code>
Unique sorted keys	<code>set</code> / <code>map</code>
Sorted keys, duplicates allowed	<code>multiset</code> / <code>multimap</code>
Fastest average lookup, no order	<code>unordered_set</code> / <code>unordered_map</code>
LIFO behavior	<code>stack</code>
FIFO behavior	<code>queue</code>
Always access largest/smallest	<code>priority_queue</code>
Bit flags, fixed size	<code>bitset</code>
Heterogeneous fixed tuple of values	<code>pair</code> / <code>tuple</code>
Optional/nullable value	<code>optional</code>
One-of-several types	<code>variant</code>

12. std::bitset (fixed-size bit sequence)

```
#include <bitset>

template<std::size_t N> class bitset;

std::bitset<8> b1;                // all zeros
std::bitset<8> b2(42);            // from unsigned long
std::bitset<8> b3("10101010");  // from string of '0'/'1'

b1.set(); b1.set(3); b1.set(3, false);
b1.reset(); b1.reset(3);
b1.flip(); b1.flip(3);
b1.test(3); b1.any(); b1.all(); b1.none();
b1.count(); b1.size();
b1.to_ulong(); b1.to_ullong(); b1.to_string();
b1 & b2; b1 | b2; b1 ^ b2; ~b1; b1 << 2; b1 >> 2;
```